

Tabla de contenido

Módulo 1 – Entender.....	1
Clase 1. Introducción a los principios de la IA Generativa.....	1
Resultados de aprendizaje observables.....	1
Parte 1.....	1
Artificial Intelligence: campo, sistema y capacidad.....	1
Machine Learning y Deep Learning.....	1
Historia e hitos de Artificial Intelligence.....	1
Parte 2.....	1
IA Generativa, Large Language Models y GPT.....	1
Conceptos de apoyo: tokens, context window, embeddings, attention y temperature.....	1
AI Fluency y 4D Framework.....	1
Evals, límites y confabulation.....	1
Casos de uso pertinentes.....	1
Limitaciones, riesgos, controversias y errores frecuentes.....	1
Buenas prácticas.....	1
Demostración sugerida para el docente.....	1
Preguntas de reflexión o comprobación.....	1
Ideas clave para convertir en diapositivas.....	1
Fuentes utilizadas en esta clase.....	1
Bibliografía.....	1

Módulo 1 – Entender

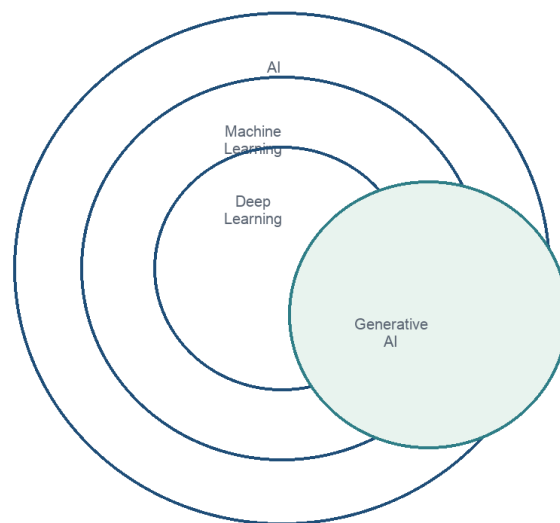
Clase 1. Introducción a los principios de la IA Generativa

Propósito. Construir un mapa conceptual sólido que permita distinguir familias de AI, comprender qué hace un Large Language Model y evaluar sus respuestas sin atribuirle capacidades humanas que no posee.

Resultados de aprendizaje observables

- Distinguir AI, Machine Learning, Deep Learning y IA Generativa con ejemplos y contraejemplos.
- Explicar intuitiva y técnicamente qué es un LLM y qué significa GPT.
- Relacionar fechas, investigadores e hitos históricos con cambios de enfoque, datos y capacidad de cómputo.
- Aplicar criterios básicos de evaluación y reconocer confabulation, límites e incertidumbre.
- Explicar AI Fluency y aplicar las cuatro competencias del 4D Framework.

Pregunta orientadora: ¿Qué ocurre realmente cuando un sistema generativo produce una respuesta convincente y cómo decidir si es confiable?



*Figura 1. Relación entre AI, ML, DL y IA Generativa. Elaboración propia con base en las fuentes citadas en la sección.
Texto alternativo: Diagrama conceptual con bloques y flechas que resume la relación central de la clase.*

Parte 1

Artificial Intelligence: campo, sistema y capacidad

Explicación intuitiva. Artificial Intelligence (AI) no es una herramienta única ni una mente digital. Es un campo de estudio y de ingeniería que reúne métodos para construir sistemas capaces de producir resultados que, en un contexto humano, asociamos con percepción, predicción, recomendación, generación de

contenido, planificación o toma asistida de decisiones. Una analogía más fiel que la de un “cerebro” es la de una familia de instrumentos: un detector de fraude, un recomendador, un sistema de visión y un asistente conversacional pueden pertenecer al campo de AI, pero resuelven problemas diferentes y no comparten necesariamente la misma arquitectura.

Conviene distinguir tres niveles. El campo de AI estudia principios, métodos y consecuencias. Un modelo es un componente matemático que transforma entradas en salidas. Un sistema de AI incluye además datos, interfaz, reglas, infraestructura, personas, permisos, procedimientos y mecanismos de supervisión. Esta distinción evita atribuirle al modelo capacidades que en realidad provienen del sistema completo, como consultar documentos, ejecutar acciones o recordar información de un usuario.

No existe una frontera académica única e inmutable entre AI y automatización avanzada. La definición de la OECD, actualizada en 2024, enfatiza que un sistema de AI infiere a partir de las entradas cómo generar predicciones, contenido, recomendaciones o decisiones que pueden influir en entornos físicos o virtuales [1]. Esta formulación es útil para gobernanza, pero no resuelve por sí sola debates filosóficos sobre inteligencia, comprensión o conciencia.

Explicación técnica. En términos de sistema, pueden identificarse entradas, una representación de la información, un mecanismo de inferencia, salidas, un objetivo y un entorno. Algunos sistemas emplean reglas simbólicas escritas por expertos; otros usan Machine Learning (ML) para ajustar modelos a partir de datos; muchos sistemas actuales combinan ambos enfoques con búsqueda, bases de datos, herramientas y controles deterministas. Por ello, AI no es sinónimo de ML y mucho menos de Large Language Model (LLM).

La palabra inferencia tiene dos usos relacionados. En ML describe la ejecución de un modelo ya ajustado sobre una entrada nueva. En la definición de la OECD se usa en un sentido más amplio: el sistema deriva cómo producir una salida a partir de entradas y objetivos. Ninguno de los dos usos implica razonamiento consciente. Autonomía tampoco es una propiedad binaria: un sistema puede recomendar, seleccionar un siguiente paso o ejecutar acciones dentro de permisos previamente definidos.

Para analizar una solución conviene preguntar dónde reside cada capacidad: qué aporta el modelo, qué información añade el contexto, qué decisiones están codificadas como reglas, qué herramientas pueden actuar sobre otros sistemas y qué persona conserva la responsabilidad. Ese mapa técnico y organizacional es más útil que discutir si la herramienta “es inteligente” en abstracto [1], [2].

Cuándo conviene y cuándo no. Conviene considerar AI cuando la tarea contiene patrones difíciles de describir con reglas exhaustivas, información no estructurada —texto, imagen, audio— o variación que exige clasificación, predicción o generación flexible. No conviene empezar por AI si una regla simple, una fórmula o una búsqueda exacta resuelve el problema con mayor estabilidad, explicabilidad y menor costo.

En decisiones de alto impacto, el uso solo es razonable si existen fuentes de verdad, evaluación representativa, controles, trazabilidad y una persona u organización responsable. La disponibilidad de un modelo no demuestra que la tarea sea apropiada para delegación.

Ejemplo correcto. Un equipo usa un clasificador para asignar una categoría preliminar a solicitudes y envía los casos ambiguos a revisión. El sistema incluye formulario, modelo, reglas de prioridad, registro, supervisor y procedimiento de corrección.

Contraejemplo o error frecuente. Llamar AI a cualquier fórmula de una hoja de cálculo o, en el extremo opuesto, suponer que un chatbot comprende el negocio como un profesional porque redacta con fluidez.

Qué puede simplificar el docente. En clase puede mostrarse el esquema entrada → procesamiento → salida. No debe omitirse que el modelo es solo un componente del sistema ni que la frontera de AI depende de la definición y el propósito de análisis.

Machine Learning y Deep Learning

Explicación intuitiva. Machine Learning (ML) es una forma de construir programas en la que no se escriben una por una todas las reglas para resolver la tarea. Se muestran datos o interacciones al sistema y un algoritmo ajusta un modelo para que mejore en una tarea definida. La formulación clásica de Tom Mitchell ayuda a evitar vaguedades: un programa aprende de una experiencia E , respecto de una tarea T y una medida de desempeño P , si su desempeño medido por P mejora con E [42]. Por ejemplo, para clasificar solicitudes, T es asignar categorías, E son ejemplos previamente clasificados y P puede ser el porcentaje de clasificaciones correctas en casos no usados para ajustar el modelo.

Deep Learning (DL) es una familia dentro de ML basada en artificial neural networks con múltiples capas de procesamiento. La idea intuitiva es que las primeras capas detectan patrones sencillos y las siguientes combinan esos patrones en representaciones más abstractas. En una imagen, una red puede pasar de bordes a texturas, luego a partes y finalmente a objetos; en lenguaje, las representaciones no corresponden tan limpiamente a categorías humanas, pero también se construyen de forma jerárquica [41].

La relación conceptual es: AI es el campo más amplio; ML es un conjunto de métodos dentro de AI; DL es una familia dentro de ML. IA Generativa suele emplear DL, pero no todo DL genera contenido y no todo sistema de AI usa ML. Un árbol de decisión para predecir cancelaciones es ML pero no DL; una red convolucional que reconoce objetos es DL pero no necesariamente IA Generativa.

Explicación técnica. Un problema de ML debe especificar datos, variables de entrada, salida esperada y una función de evaluación. En supervised learning, los ejemplos incluyen una etiqueta o valor objetivo; en unsupervised learning se buscan estructuras sin una etiqueta objetivo, como agrupaciones; en self-supervised learning las señales de aprendizaje se construyen a partir de los propios datos, como predecir partes ocultas de un texto; en reinforcement learning un agente aprende una política mediante recompensas asociadas a secuencias de acciones. Estas categorías se solapan en sistemas modernos y no constituyen una taxonomía perfecta.

Un modelo contiene parámetros ajustables. Durante training, un algoritmo compara predicciones con objetivos mediante una loss function y modifica los parámetros para reducir el error. En artificial neural networks, backpropagation calcula cómo contribuye cada parámetro al error y un optimizer actualiza los pesos. Durante inference, los parámetros se mantienen normalmente fijos y el modelo calcula una salida para datos nuevos. Esta explicación debe aparecer después de definir ML y DL, porque training e inference son fases operativas, no definiciones del campo [38], [41].

La meta no es memorizar el conjunto de training, sino generalizar a casos no observados. Por eso se separan datos de training, validation y test. Overfitting ocurre cuando el modelo se adapta demasiado a particularidades del conjunto de training y empeora fuera de él. Distribution shift aparece cuando la población o el proceso real cambia respecto de los datos usados para construir el modelo. Un buen resultado promedio tampoco garantiza desempeño adecuado para todos los subgrupos.

DL redujo la necesidad de diseñar manualmente algunas features porque aprende representaciones a partir de datos, pero no elimina decisiones humanas: selección y documentación de datos, arquitectura, objetivo,

métricas, umbrales y condiciones de despliegue. Sus ventajas suelen exigir más datos, cómputo y evaluación que métodos más simples [40], [41].

Cuándo conviene y cuándo no. ML conviene para tareas repetibles con ejemplos suficientes y una medida de desempeño estable: clasificación, predicción, detección de anomalías o ranking. DL resulta especialmente útil en texto, imagen, audio y señales complejas, cuando el volumen y la diversidad de datos justifican el costo. No conviene un modelo complejo si una regla, consulta o modelo estadístico sencillo cumple el objetivo con menos riesgo.

Un LLM no debe usarse como calculadora o base transaccional por defecto. Puede explicar una hoja de cálculo o producir una fórmula propuesta, pero un motor de cálculo debe ejecutar y verificar las operaciones críticas.

Ejemplo correcto. Para estimar demanda, el equipo define el horizonte, reúne historial y variables conocidas, separa un periodo de test y compara el modelo contra una baseline simple. La mejora se mide en datos no utilizados durante training.

Contraejemplo o error frecuente. Afirmar que el modelo “aprendió el negocio” porque disminuyó la loss de training, o reportar precisión de los mismos datos usados para ajustarlo.

Qué puede simplificar el docente. Puede omitirse la derivación matemática de gradient descent y backpropagation. Deben conservarse las ideas de tarea-experiencia-métrica, relación $AI \supset ML \supset DL$, separación training/inference, generalization y posibilidad de overfitting.

Historia e hitos de Artificial Intelligence

Explicación intuitiva. La historia de AI no empieza con ChatGPT y tampoco sigue una línea recta hacia una inteligencia cada vez más humana. Es una sucesión de preguntas, enfoques y ciclos de entusiasmo: ¿qué puede calcular una máquina?, ¿puede una conducta lingüística servir como evidencia de inteligencia?, ¿conviene representar conocimiento con símbolos o aprender patrones de datos?, ¿qué cambia cuando aumentan el cómputo, los datos y la escala? Entender esa secuencia permite explicar por qué hoy conviven reglas, ML, DL, búsqueda y modelos generativos.

Los hitos deben interpretarse por la limitación que intentaron resolver, no como una lista de productos. Algunos aportaron fundamentos teóricos; otros mostraron que una técnica podía funcionar a escala; otros revelaron límites y provocaron correcciones de expectativas. La expresión “AI winter” describe periodos de financiación e interés reducidos, pero sus fechas, causas y alcance varían entre relatos históricos; no hay una cronología única aceptada [39].

Explicación técnica. 1936–1937 — Alan Turing formalizó una máquina abstracta capaz de ejecutar procedimientos paso a paso y mostró límites de lo computable en “On Computable Numbers” [33]. Este trabajo no creó AI, pero estableció una base de Computer Science: antes de preguntar si una máquina puede comportarse inteligentemente, hay que precisar qué significa computar un procedimiento.

1943 — Warren McCulloch y Walter Pitts propusieron un modelo lógico de neuronas conectadas [36]. Fue una idealización matemática, no una reproducción fiel del cerebro, pero abrió una línea de investigación sobre computation distribuida en redes. En 1950, Alan Turing publicó “Computing Machinery and Intelligence” y sustituyó la pregunta ambigua “¿pueden pensar las máquinas?” por el imitation game, una prueba conductual basada en conversación escrita [34]. El llamado Turing Test evalúa indistinguibilidad en una interacción; no demuestra conciencia, verdad, comprensión general ni seguridad.

1955–1956 — John McCarthy, Marvin Minsky, Nathaniel Rochester y Claude Shannon redactaron el 31 de agosto de 1955 la propuesta del Dartmouth Summer Research Project on Artificial Intelligence, celebrado en el verano de 1956. La propuesta es generalmente reconocida como el origen del término “artificial intelligence” para un programa de investigación organizado [35], [39]. En 1958, Frank Rosenblatt presentó el perceptron, un modelo que ajustaba pesos para clasificar patrones [37]. Sus capacidades eran limitadas, pero estableció una conexión duradera entre aprendizaje y artificial neural networks.

Décadas de 1960–1980 — El symbolic AI representó conocimiento mediante símbolos, reglas, búsqueda y lógica. Programas como ELIZA demostraron que una conversación superficial podía inducir atribuciones humanas sin comprensión. Los expert systems de los años setenta y ochenta codificaron conocimiento especializado y lograron valor en dominios acotados, pero eran costosos de mantener y frágiles ante situaciones no previstas. Las expectativas excesivas, límites de hardware, datos y generalización contribuyeron a periodos denominados AI winters [39].

1986–2012 — David Rumelhart, Geoffrey Hinton y Ronald Williams popularizaron un procedimiento práctico de backpropagation para aprender representaciones internas en redes multicapa [38]. Durante las décadas siguientes, el avance combinó algoritmos, datos digitalizados, GPUs y mejores prácticas de training. En 2012, Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton obtuvieron un salto de desempeño en ImageNet con una deep convolutional neural network entrenada en GPU, un hito para la adopción moderna de DL en visión [40], [41].

2017–2022 — Ashish Vaswani y coautores presentaron Transformer, una arquitectura basada en attention que facilitó el procesamiento paralelo de secuencias [3]. En 2018 apareció el primer GPT de OpenAI; GPT-3 mostró en 2020 capacidades de few-shot learning a gran escala [4]. Desde 2022, interfaces conversacionales hicieron accesibles estos modelos a públicos masivos. Este salto de adopción no implica que se hayan resuelto confabulation, bias, actualización de conocimiento, derechos de autor, costo o control [5].

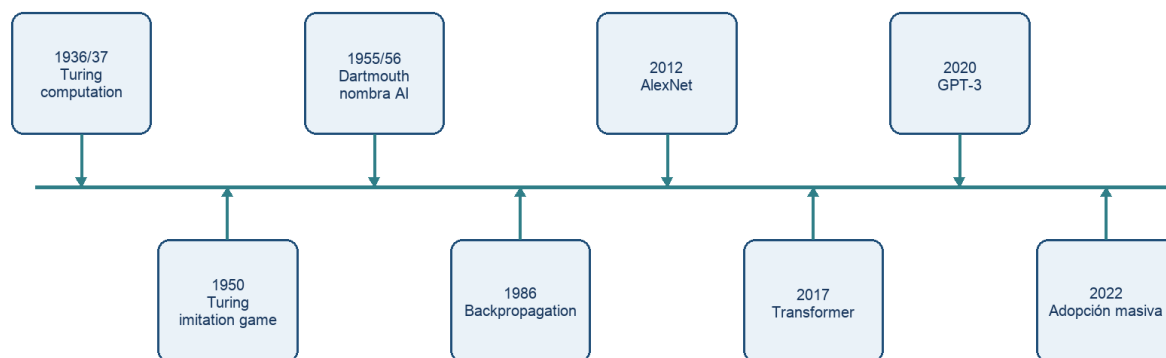
Cuándo conviene y cuándo no. La historia conviene para explicar por qué un sistema actual combina ideas de distintas épocas y para moderar dos narrativas incorrectas: que todo avance reciente era inevitable y que aumentar escala resolverá automáticamente cualquier problema. No conviene usar una cronología comercial como sustituto de la historia científica ni atribuir una invención compleja a una sola persona.

También ayuda a distinguir cambio de paradigma y cambio de interfaz. La experiencia conversacional es nueva para muchos usuarios, pero se apoya en décadas de language modeling, neural networks, optimization y Human–Computer Interaction.

Ejemplo correcto. Presentar 1936/37, 1943, 1950, 1955/56, 1958, 1986, 2012, 2017, 2020 y 2022; en cada fecha explicar el problema abordado, el aporte y el límite que permaneció.

Contraejemplo o error frecuente. Afirmar que Alan Turing “inventó AI”, que Dartmouth produjo de inmediato sistemas inteligentes o que AlexNet/Transformer fueron descubrimientos aislados sin antecedentes. También es incorrecto decir que un chatbot superó el Turing Test y, por ello, comprende o piensa como una persona.

Qué puede simplificar el docente. La presentación puede elegir ocho hitos. La guía docente debe conservar nombres, fechas y matices, especialmente la diferencia entre fundamento de computation, nacimiento del campo, resurgimiento de neural networks y adopción de IA Generativa.



⚠ hitos cambian métodos y escala; no forman una ruta inevitable hacia inteligencia humana.

Figura 2. Hitos seleccionados de la historia de AI. Elaboración propia con base en [3], [33]–[41]. Texto alternativo: Línea de tiempo con 1936/37 Turing y computation, 1950 imitation game, 1955/56 Dartmouth, 1986 backpropagation, 2012 AlexNet, 2017 Transformer, 2020 GPT-3 y 2022 adopción masiva.

Parte 2

IA Generativa, Large Language Models y GPT

Explicación intuitiva. IA Generativa designa sistemas que producen contenido nuevo —texto, imagen, audio, video o código— a partir de patrones aprendidos. “Nuevo” no significa creado desde la nada: el sistema recombina regularidades estadísticas de training data y del contexto actual. Un clasificador que decide si un correo es urgente es IA discriminativa; un sistema que redacta una respuesta propuesta es IA Generativa. Muchos productos combinan ambas capacidades.

Un Large Language Model (LLM) es un modelo entrenado para representar y generar lenguaje mediante secuencias de tokens. Puede imaginarse como un sistema de predicción contextual extremadamente sofisticado: dado lo disponible en su context window, asigna probabilidades a continuaciones posibles. La analogía de autocomplete es útil para introducirlo, pero es incompleta: un LLM aprende representaciones que permiten resumir, traducir formatos, seguir instrucciones y resolver ciertos patrones no programados de forma explícita.

GPT significa Generative Pre-trained Transformer. Generative indica que produce secuencias; Pre-trained, que adquiere capacidades generales mediante training previo sobre grandes colecciones antes de adaptarse a instrucciones o tareas; Transformer identifica la familia de arquitectura introducida en 2017. GPT es una familia concreta de modelos, no un sinónimo de LLM ni de chatbot.

Explicación técnica. Un language model asigna una probabilidad a una secuencia o, de forma autoregresiva, estima la distribución del siguiente token condicionada por los tokens anteriores. La respuesta se genera repitiendo el ciclo: tokenización, cálculo de representaciones, distribución de probabilidades, selección de un token y nueva evaluación con el token añadido. El modelo no compone necesariamente primero una respuesta completa ni consulta una base factual, salvo que el sistema le proporcione herramientas o fuentes.

En pre-training, muchos LLM optimizan next-token prediction sobre grandes corpus. Esa señal self-supervised permite aprender regularidades lingüísticas y patrones de conocimiento presentes en los datos. Posteriormente puede existir post-training mediante supervised fine-tuning, preference optimization u otras técnicas para mejorar instruction following, seguridad y comportamiento. Los detalles cambian por modelo y proveedor y con frecuencia no se publican por completo; por ello no debe afirmarse una receta exacta sin documentación [3], [4].

Muchos GPT son decoder-only Transformers: cada capa combina self-attention, transformaciones feed-forward, residual connections y normalization. Self-attention permite que cada posición pondere información de otras posiciones autorizadas por la causal mask. Multi-head attention aprende distintas relaciones en paralelo. Positional information conserva el orden, porque attention por sí sola no representa secuencia. El resultado final se proyecta sobre el vocabulary para obtener logits y probabilidades del siguiente token [3].

La fluidez no es un módulo de verificación de hechos. Next-token prediction favorece continuaciones plausibles, no necesariamente verdaderas. El conocimiento paramétrico puede ser incompleto, desactualizado o inconsistente; el contexto puede contener errores; y sampling introduce variación. Grounding, Retrieval-Augmented Generation (RAG), tool use y evals reducen algunos riesgos, pero no convierten el sistema en una fuente infalible [5].

Cuándo conviene y cuándo no. Conviene usar un LLM para transformar información no estructurada, generar borradores, resumir con fuentes, clasificar texto, proponer estructuras y facilitar exploración. No conviene usarlo como única fuente para hechos actuales, referencias, cifras críticas o decisiones de alto impacto.

Cuando la tarea requiere cálculo exacto, consulta transaccional o datos vigentes, el LLM puede coordinar la interacción, pero la operación debe ejecutarse en una calculadora, base de datos o API y validarse antes de presentarse.

Ejemplo correcto. Suministrar un informe aprobado, pedir un resumen para una audiencia definida, exigir citas por página y contrastar cada afirmación con el documento antes de publicar.

Contraejemplo o error frecuente. Pedir bibliografía sin fuentes ni navegación y asumir que cada título existe; o tratar la explicación verbal de una suma como prueba de que el cálculo es correcto.

Qué puede simplificar el docente. Puede usarse “autocomplete contextual a gran escala” como primera imagen. Deben conservarse token-by-token generation, pre-training/post-training, Transformer, context dependence y la separación entre fluidez y verdad.

Conceptos de apoyo: tokens, context window, embeddings, attention y temperature

Pertinencia como concepto de apoyo. Son pertinentes porque explican cómo entra la información al LLM, por qué el contexto es limitado, cómo se relacionan fragmentos, cómo funciona la búsqueda semántica y por qué dos respuestas pueden variar.

Explicación intuitiva. Un token es una unidad que el modelo procesa: puede ser una palabra corta, una parte de palabra, signos o espacios. El LLM no recibe directamente “ideas” ni páginas; recibe identificadores de tokens. La context window es su mesa de trabajo temporal: contiene system instructions, conversación, documentos recuperados, resultados de herramientas y el texto que se está generando. Si la mesa se llena o se contamina con información irrelevante, el desempeño puede empeorar.

Un embedding representa un fragmento como un vector en un espacio numérico donde cierta cercanía captura similitud aprendida. Es útil imaginarlo como un mapa semántico, aunque no exista un eje humano para cada significado. Attention es el mecanismo que permite ponderar relaciones entre posiciones del contexto. Temperature modifica cómo se distribuye el sampling entre continuaciones posibles: valores más bajos suelen concentrar las elecciones, pero no verifican la respuesta.

Explicación técnica. Tokenization segmenta la entrada con un vocabulary finito y convierte cada token en un identificador. Una matriz de embeddings lo proyecta a un vector denso. Como los límites de la context window se expresan en tokens, longitud en páginas o palabras es solo una aproximación; idioma, formato y código alteran el conteo.

En scaled dot-product attention, cada posición produce vectores query, key y value. La similitud entre query y keys genera pesos normalizados que combinan values. Multi-head attention repite el mecanismo en subespacios distintos. En un LLM autoregressive, una causal mask impide que una posición use tokens futuros durante generation [3]. Attention no debe interpretarse automáticamente como una explicación causal de la decisión del modelo.

Para generation, los logits se transforman en probabilidades. Temperature divide los logits antes de softmax: reducirla concentra masa en opciones de mayor puntuación; aumentarla distribuye más probabilidad entre alternativas. Otros controles como top-p o top-k también afectan sampling. Incluso con temperature baja puede haber variación por infraestructura, cambios de modelo o operaciones no deterministas, y una salida estable puede seguir siendo falsa.

Los embeddings utilizados para retrieval no son la misma cosa que la memoria del chat. Se calculan para consultas y fragmentos, luego una métrica de similitud ayuda a seleccionar candidatos. El resultado depende del modelo de embeddings, chunking, metadata, filtros y ranking [21].

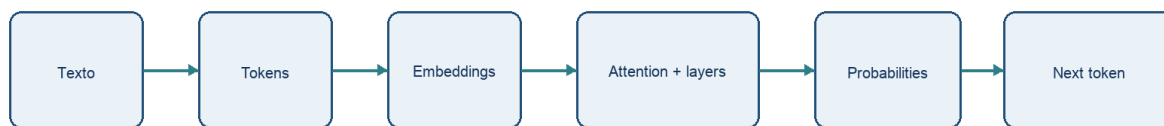
Cuándo conviene y cuándo no. Conviene administrar el contexto con selección, jerarquía, delimitación y citas. Embeddings convienen para búsqueda semántica cuando las palabras exactas pueden variar. Temperature más alta puede ser útil para ideación; más baja, para formatos y tareas consistentes. Ninguna configuración reemplaza evals.

No conviene cargar documentos indiscriminadamente ni asumir que el máximo context window garantiza mejor respuesta. Contexto largo puede añadir ruido, contradicciones, instrucciones hostiles y costo.

Ejemplo correcto. Recuperar solo los fragmentos de política pertinentes, con título, fecha y página, y pedir una respuesta limitada a esos fragmentos.

Contraejemplo o error frecuente. Subir cien documentos sin filtros y concluir que el modelo “los aprendió”; o bajar temperature a cero y presentar la salida como verdad verificada.

Qué puede simplificar el docente. Pueden omitirse matrices y softmax. Deben conservarse token como unidad de procesamiento, context window como recurso finito, embeddings para cercanía semántica, attention como ponderación contextual y temperature como control de sampling, no de exactitud.



El ciclo se repite token a token; el contexto condiciona cada predicción.

Figura 3. Tokens, context window y generation iterativa. Elaboración propia con base en las fuentes citadas en la sección. Texto alternativo: Flujo de texto a tokens, embeddings, attention, probabilities y next token; el ciclo se repite.

AI Fluency y 4D Framework

Explicación intuitiva. AI Fluency es la capacidad de interactuar con sistemas de AI de manera efectiva, eficiente, ética y segura. El 4D Framework de Rick Dakan, Joseph Feller y Anthropic organiza esa fluidez en cuatro competencias conectadas: Delegation, Description, Discernment y Diligence [31], [32]. Es un marco educativo y profesional útil, no un estándar internacional ni un consenso académico obligatorio.

Delegation pregunta qué trabajo debe hacer la persona, qué parte puede apoyar AI y en qué modo conviene colaborar. Description consiste en comunicar objetivos, proceso y criterios de desempeño. Discernment exige evaluar tanto el output como el proceso y el comportamiento del sistema. Diligence recuerda que la persona conserva responsabilidad sobre creación, transparencia y deployment.

El marco también distingue tres modos de interacción: Automation, cuando AI ejecuta una tarea indicada; Augmentation, cuando persona y AI colaboran; y Agency, cuando se configura un sistema para actuar posteriormente con mayor independencia. Estos modos no son niveles de superioridad: la selección depende del riesgo, la tarea y la capacidad de supervisión [31].

Explicación técnica. Delegation puede analizarse mediante problem awareness, platform awareness y task delegation: comprender el problema, conocer capacidades y límites de la plataforma, y asignar trabajo apropiado. Description se descompone en product, process y performance description: especificar resultado, forma de trabajo y criterios de éxito. Discernment aplica esas mismas tres perspectivas para revisar output, proceso y comportamiento. Diligence cubre creation, transparency y deployment diligence [31].

La utilidad del 4D Framework está en conectar prompting con decisiones anteriores y posteriores. Un prompt excelente no corrige una mala delegación; una respuesta convincente no elimina Discernment; una mejora de productividad no reemplaza Diligence. Así, Prompt Engineering es solo una parte de Description y del ciclo más amplio de AI Fluency.

Cuándo conviene y cuándo no. Conviene usar el marco para preparar explicaciones sobre colaboración profesional con AI y para organizar responsabilidades. No conviene presentarlo como una certificación universal, una teoría exhaustiva de Human–AI Interaction o evidencia de que un producto es seguro.

Su fuerza docente es ofrecer preguntas estables aunque cambien las interfaces: qué delegar, cómo describir, cómo evaluar y cómo responder por el uso.

Ejemplo correcto. Para preparar un presupuesto, la persona conserva supuestos y aprobación (Delegation), describe estructura y fuentes (Description), revisa fórmulas y cifras (Discernment) y documenta el uso de AI y protege los datos (Diligence).

Contraejemplo o error frecuente. Reducir AI Fluency a saber escribir prompts o asumir que usar Agency siempre es más avanzado que Augmentation.

Qué puede simplificar el docente. En diapositivas puede mostrarse una pregunta por cada D. La guía debe conservar que es un marco de autores específicos, su licencia CC BY-NC-SA 4.0 y la relación con Automation, Augmentation y Agency.

Evals, límites y confabulation

Explicación intuitiva. Evaluar una respuesta de IA Generativa no consiste en decidir si “suena profesional”. La pregunta correcta es si cumple una tarea definida bajo criterios observables. Una respuesta puede tener buen tono y fallar en hechos, cobertura o seguridad; otra puede ser correcta pero inútil para la audiencia. Por eso los evals deben separar dimensiones como factualidad, fidelidad a fuentes, cobertura, formato, utilidad, bias, costo y latencia.

NIST usa el término confabulation para salidas falsas o erróneas presentadas con confianza; en lenguaje común se usa hallucination [5]. No hay una taxonomía académica única ni una tasa universal. La frecuencia depende del modelo, la tarea, el contexto, las herramientas y el método de evaluación. Una cifra aislada no debe extrapolarse a todos los usos.

Explicación técnica. Un eval comienza con casos representativos y casos adversariales, una expected behavior y criterios de puntuación. En tareas cerradas puede existir una respuesta exacta. En tareas abiertas se usan rúbricas, revisión humana, comparación por pares o model-based grading, pero cada método introduce error. Pedir al mismo modelo que juzgue su salida no constituye validación independiente.

Los evals deben ejecutarse sobre versiones concretas de prompt, modelo, herramientas y fuentes. Una media oculta errores críticos; por ello se analizan distribuciones, segmentos y failure modes. También debe compararse contra una baseline: proceso manual, regla simple o versión anterior. El conjunto de test no debe convertirse gradualmente en material de ajuste sin mantener un holdout.

Los límites incluyen conocimiento desactualizado, confabulation, sensitivity al prompt, bias, context degradation, falta de calibration, vulnerabilidad a prompt injection y errores de tool use. Grounding y RAG reducen algunos errores factuales, pero retrieval puede traer un fragmento equivocado y generation puede interpretarlo mal [5], [16].

Cuándo conviene y cuándo no. Conviene usar LLM como borrador o apoyo cuando el resultado es verificable y el control es proporcional al impacto. No conviene delegar decisiones legales, clínicas, financieras o laborales sin gobernanza, evidencia, trazabilidad y responsable competente.

En productos, un eval inicial no sustituye monitoring. Los datos, usuarios, proveedores y modelos cambian; un sistema que funcionó en un piloto puede degradarse.

Ejemplo correcto. Crear 40 casos de resúmenes reales anonimizados, puntuar fidelidad y cobertura por separado, incluir casos con información ausente y exigir abstención cuando no exista evidencia.

Contraejemplo o error frecuente. Evaluar cinco prompts elegidos por el autor, reportar solo los mejores ejemplos o usar estilo como proxy de exactitud.

Qué puede simplificar el docente. Puede presentarse una matriz impacto del error × facilidad de verificación. Deben conservarse la necesidad de baseline, casos representativos, análisis de errores y separación entre calidad lingüística y factualidad.

Casos de uso pertinentes

- Preparar borradores de presentaciones a partir de fuentes verificadas.
- Organizar documentos y extraer temas con revisión humana.
- Explicar una hoja de cálculo sin delegar cálculos críticos al lenguaje generativo.

Limitaciones, riesgos, controversias y errores frecuentes

- Antropomorfismo: confundir lenguaje natural con conciencia o intención.
- Jerarquía incorrecta: dibujar AI dentro de ML; la relación programada es $AI \supset ML \supset DL$.
- Actualidad: asumir que el conocimiento paramétrico está actualizado.
- Evaluación superficial: medir estilo cuando el riesgo principal es exactitud.

Buenas prácticas

- Definir tarea, usuario, fuente de verdad y criterio de éxito antes de escoger herramienta.
- Separar borrador, verificación y aprobación.
- Registrar versión, fecha y configuración cuando se comparan resultados.
- Usar modelos más simples o reglas cuando resuelven el problema con menor riesgo.

Demostración sugerida para el docente

Comparar una pregunta factual sin fuentes y la misma pregunta con un fragmento verificable. Mostrar tokens aproximados, pedir citas y evidenciar que una respuesta fluida puede requerir verificación. No usar datos reales sensibles.

Preguntas de reflexión o comprobación

- ¿Qué diferencia un modelo de un sistema de AI completo?
- ¿Por qué un LLM puede ser fluido y estar equivocado?
- ¿En qué caso una regla sería preferible a ML?
- ¿Qué criterio de evaluación cambia según el impacto del error?

Ideas clave para convertir en diapositivas

- Mapa AI → ML → DL y solapamiento de IA Generativa.
- Línea de tiempo con fechas, investigadores, aportes y límites.
- LLM como predicción iterativa de tokens.
- Fluidez ≠ verdad; modelo ≠ sistema.
- AI Fluency: Delegation, Description, Discernment y Diligence.
- Evaluación proporcional al riesgo.

Fuentes utilizadas en esta clase

Referencias principales: [1], [2], [3], [4], [5], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [42].

Las características de productos citadas deben verificarse de nuevo antes de impartir la sesión, porque pueden cambiar con versiones, planes y región.

Bibliografía

Fecha de consulta de todas las fuentes web: 28 de agosto de 2026. Los enlaces se verificaron como destinos directos durante la preparación del documento. Cuando la fuente no presenta fecha de publicación o actualización, se omite ese dato conforme al criterio solicitado.

- [1] OECD, “Explanatory memorandum on the updated OECD definition of an AI system,” OECD Artificial Intelligence Papers, no. 8, Mar. 2024. [En línea]. Disponible: <https://doi.org/10.1787/623da898-en>. [Consultado: 28-ago-2026].
- [2] S. Russell and P. Norvig, “Artificial Intelligence: A Modern Approach, 4th ed.,” Pearson, 2021. [En línea]. Disponible: <https://aima.cs.berkeley.edu/>. [Consultado: 28-ago-2026].
- [3] A. Vaswani et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems 30, 2017. [En línea]. Disponible: <https://arxiv.org/abs/1706.03762>. [Consultado: 28-ago-2026].
- [4] T. B. Brown et al., “Language Models are Few-Shot Learners,” Advances in Neural Information Processing Systems 33, 2020. [En línea]. Disponible: <https://papers.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>. [Consultado: 28-ago-2026].
- [5] NIST, “Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1,” National Institute of Standards and Technology, Jul. 2024; updated Apr. 2026. [En línea]. Disponible: <https://doi.org/10.6028/NIST.AI.600-1>. [Consultado: 28-ago-2026].
- [6] Stanford Institute for Human-Centered AI, “The 2026 AI Index Report,” Stanford University, 2026. [En línea]. Disponible: <https://hai.stanford.edu/ai-index/2026-ai-index-report>. [Consultado: 28-ago-2026].
- [7] E. Brynjolfsson, D. Li and L. R. Raymond, “IA Generativa at Work,” NBER Working Paper 31 161, Apr. 2023, rev. Oct. 2023. [En línea]. Disponible: <https://www.nber.org/papers/w31161>. [Consultado: 28-ago-2026].
- [8] UNESCO, “Recommendation on the Ethics of Artificial Intelligence,” UNESCO, Nov. 2021. [En línea]. Disponible: <https://www.unesco.org/en/legal-affairs/recommendation-ethics-artificial-intelligence>. [Consultado: 28-ago-2026].
- [9] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 (Artificial Intelligence Act),” Official Journal of the European Union, Jun. 2024. [En línea]. Disponible: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>. [Consultado: 28-ago-2026].
- [10] NIST, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” National Institute of Standards and Technology, Jan. 2023. [En línea]. Disponible: <https://doi.org/10.6028/NIST.AI.100-1>. [Consultado: 28-ago-2026].
- [11] OpenAI, “Prompt engineering,” OpenAI API Documentation. [En línea]. Disponible: <https://developers.openai.com/api/docs/guides/prompt-engineering>. [Consultado: 28-ago-2026].
- [12] Google, “Prompt design strategies,” Google AI for Developers. [En línea]. Disponible: <https://ai.google.dev/gemini-api/docs/prompting-strategies>. [Consultado: 28-ago-2026].
- [13] Anthropic, “Prompt engineering overview,” Claude Platform Docs. [En línea]. Disponible: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>. [Consultado: 28-ago-2026].

- [14] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” Advances in Neural Information Processing Systems 35, 2022. [En línea]. Disponible: <https://proceedings.neurips.cc/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html>. [Consultado: 28-ago-2026].
- [15] T. Kojima et al., “Large Language Models are Zero-Shot Reasoners,” Advances in Neural Information Processing Systems 35, 2022. [En línea]. Disponible: <https://proceedings.neurips.cc/paper/2022/hash/8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html>. [Consultado: 28-ago-2026].
- [16] OpenAI, “Evaluation best practices,” OpenAI API Documentation. [En línea]. Disponible: <https://developers.openai.com/api/docs/guides/evaluation-best-practices>. [Consultado: 28-ago-2026].
- [17] Google, “Multimodal concepts,” Google AI for Developers. [En línea]. Disponible: <https://ai.google.dev/gemini-api/docs>. [Consultado: 28-ago-2026].
- [18] NIST, “Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management, Version 1.0,” National Institute of Standards and Technology, Jan. 2020. [En línea]. Disponible: <https://doi.org/10.6028/NIST.CSWP.01162020>. [Consultado: 28-ago-2026].
- [19] OWASP Foundation, “OWASP Top 10 for LLM Applications 2025,” OWASP GenAI Security Project, 2025. [En línea]. Disponible: <https://genai.owasp.org/llm-top-10/>. [Consultado: 28-ago-2026].
- [20] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems 33, 2020. [En línea]. Disponible: <https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>. [Consultado: 28-ago-2026].
- [21] OpenAI, “Vector embeddings,” OpenAI API Documentation. [En línea]. Disponible: <https://developers.openai.com/api/docs/guides/embeddings>. [Consultado: 28-ago-2026].
- [22] Microsoft, “Available knowledge sources for agents,” Microsoft Learn, Jul. 2026. [En línea]. Disponible: <https://learn.microsoft.com/en-us/microsoft-copilot-studio/agents-experience/knowledge-sources-overview>. [Consultado: 28-ago-2026].
- [23] Microsoft, “Orchestrate agent behavior with IA Generativa,” Microsoft Learn, 2026. [En línea]. Disponible: <https://learn.microsoft.com/en-us/microsoft-copilot-studio/advanced-generative-actions>. [Consultado: 28-ago-2026].
- [24] Google, “Function calling with the Gemini API,” Google AI for Developers. [En línea]. Disponible: <https://ai.google.dev/gemini-api/docs/function-calling>. [Consultado: 28-ago-2026].
- [25] OpenAI, “Function calling,” OpenAI API Documentation. [En línea]. Disponible: <https://developers.openai.com/api/docs/guides/function-calling>. [Consultado: 28-ago-2026].
- [26] Zapier, “Zap workflows quick start guide,” Zapier Help Center, Jun. 2026. [En línea]. Disponible: <https://help.zapier.com/hc/en-us/articles/22234847450893-Zap-workflows-quick-start-guide>. [Consultado: 28-ago-2026].
- [27] n8n, “Workflows,” n8n Documentation. [En línea]. Disponible: <https://docs.n8n.io/workflows/>. [Consultado: 28-ago-2026].
- [28] ISO, “ISO/IEC 42001:2023 Information technology — Artificial intelligence — Management system,” ISO, Dec. 2023. [En línea]. Disponible: <https://www.iso.org/standard/81230.html>. [Consultado: 28-ago-2026].

- [29] H. Booth et al., “Secure Software Development Practices for IA Generativa and Dual-Use Foundation Models: An SSDF Community Profile, NIST SP 800-218A,” National Institute of Standards and Technology, Jul. 2024. [En línea]. Disponible: <https://doi.org/10.6028/NIST.SP.800-218A>. [Consultado: 28-ago-2026].
- [30] OWASP Foundation, “LLM02:2025 Sensitive Information Disclosure,” OWASP GenAI Security Project, 2025. [En línea]. Disponible: <https://genai.owasp.org/llmrisk/llm022025-sensitive-information-disclosure/>. [Consultado: 28-ago-2026].
- [31] R. Dakan, J. Feller and Anthropic, “AI Fluency: Key Terminology Cheat Sheet,” Anthropic, 2025; CC BY-NC-SA 4.0. [Documento local]. Disponible en: REFERENCIAS/AI_Fluency_vocabulary_cheat_sheet.pdf. [Consultado: 28-ago-2026].
- [32] Anthropic, “AI Fluency: Frameworks and Foundations,” Anthropic. [En línea]. Disponible: <https://www.anthropic.com/ai-fluency/overview>. [Consultado: 28-ago-2026].
- [33] A. M. Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem,” Proceedings of the London Mathematical Society, ser. 2, vol. 42, no. 1, pp. 230–265, 1937. [En línea]. Disponible: <https://doi.org/10.1112/plms/s2-42.1.230>. [Consultado: 28-ago-2026].
- [34] A. M. Turing, “Computing Machinery and Intelligence,” Mind, vol. LIX, no. 236, pp. 433–460, Oct. 1950. [En línea]. Disponible: <https://doi.org/10.1093/mind/LIX.236.433>. [Consultado: 28-ago-2026].
- [35] J. McCarthy, M. L. Minsky, N. Rochester and C. E. Shannon, “A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence,” Dartmouth College, Aug. 31, 1955. [En línea]. Disponible: <https://www-formal.stanford.edu/jmc/history/dartmouth/dartmouth.html>. [Consultado: 28-ago-2026].
- [36] W. S. McCulloch and W. Pitts, “A Logical Calculus of the Ideas Immanent in Nervous Activity,” Bulletin of Mathematical Biophysics, vol. 5, pp. 115–133, Dec. 1943. [En línea]. Disponible: <https://doi.org/10.1007/BF02478259>. [Consultado: 28-ago-2026].
- [37] F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” Psychological Review, vol. 65, no. 6, pp. 386–408, 1958. [En línea]. Disponible: <https://doi.org/10.1037/h0042519>. [Consultado: 28-ago-2026].
- [38] D. E. Rumelhart, G. E. Hinton and R. J. Williams, “Learning Representations by Back-Propagating Errors,” Nature, vol. 323, pp. 533–536, Oct. 1986. [En línea]. Disponible: <https://doi.org/10.1038/323533a0>. [Consultado: 28-ago-2026].
- [39] Stanford University, “Appendix I: A Short History of AI,” One Hundred Year Study on Artificial Intelligence (AI100), 2016. [En línea]. Disponible: https://ai100.stanford.edu/sites/g/files/sbiybj18871/files/media/file/ai100report10032016fnl_singles.pdf. [Consultado: 28-ago-2026].
- [40] A. Krizhevsky, I. Sutskever and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” Advances in Neural Information Processing Systems 25, 2012. [En línea]. Disponible: https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html. [Consultado: 28-ago-2026].
- [41] Y. LeCun, Y. Bengio and G. Hinton, “Deep Learning,” Nature, vol. 521, pp. 436–444, May 2015. [En línea]. Disponible: <https://doi.org/10.1038/nature14539>. [Consultado: 28-ago-2026].

- [42] T. M. Mitchell, “Machine Learning,” McGraw-Hill, 1997. [En línea]. Disponible: <https://www.cs.cmu.edu/~tom/mlbook.html>. [Consultado: 28-ago-2026].
- [43] S. Schulhoff et al., “The Prompt Report: A Systematic Survey of Prompting Techniques,” arXiv:2406.06608, Jun. 2024. [En línea]. Disponible: <https://arxiv.org/abs/2406.06608>. [Consultado: 28-ago-2026].
- [44] Anthropic, “Effective Context Engineering for AI Agents,” Anthropic Engineering, Sep. 29, 2025. [En línea]. Disponible: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>. [Consultado: 28-ago-2026].
- [45] L. Mei et al., “A Survey of Context Engineering for Large Language Models,” arXiv:2507.13334, Jul. 2025. [En línea]. Disponible: <https://arxiv.org/abs/2507.13334>. [Consultado: 28-ago-2026].